Abstract

This report presents a detailed technical methodology and in-depth analysis of a simulation of the BB84 Quantum Key Distribution (QKD) protocol, with a specific focus on the realistic modeling of quantum channel noise and its impact on protocol performance. Moving beyond a basic proof-of-concept, this project's core objective is to quantify the performance degradation caused by intrinsic channel imperfections and transmission distance, and to rigorously validate the protocol's security guarantees under these non-ideal conditions. A sophisticated simulation framework was constructed using Python and the Qiskit-Aer library, featuring advanced noise models that include Amplitude Damping, Phase Damping (Dephasing), and generic Depolarizing errors.

The methodology section provides a thorough breakdown of the mathematical and physical principles behind these noise models, including a first-principles derivation of the distance-dependent photon loss based on the Beer-Lambert Law. It also details the information-theoretic foundations that govern the protocol's security, such as the derivation of the ~11% QBER threshold from the Shannon binary entropy function, H(Q).

The results are presented through a series of multi-dimensional visualizations. Three-dimensional surface plots reveal the intricate relationship between transmission distance, intrinsic channel noise, and the resulting Quantum Bit Error Rate (QBER), both with and without the presence of an eavesdropper. The analysis confirms that while environmental factors systematically increase the baseline QBER, they do not compromise the protocol's ability to detect an intercept-resend attack, whose QBER signature remains statistically distinct. Further analysis quantifies the severe impact of distance on the Secure Key Rate (SKR), demonstrating the exponential decay in efficiency. This report provides a robust, quantitative validation of BB84's security and a clear exposition of the physical challenges inherent in its real-world deployment.

Table of Contents

1. Project Introduction and Objectives
1.1. Problem Statement
1.2. Core Objectives
1.3. Scope and Delimitations

2. Theoretical Framework: Modeling the Noisy Quantum Channel
2.1. The Ideal vs. The Real Quantum Channel
2.2. Amplitude Damping: The Physics of Photon Loss
2.2.1. Physical Mechanism: Energy Relaxation and Absorption
2.2.2. Mathematical Model: The Beer-Lambert Law
2.2.3. Engineering Formulation: Attenuation in Decibels (dB)

2.2.4. Implementation in the Simulation
2.3. Phase Damping: The Loss of Quantum Coherence
2.3.1. Physical Mechanism: Dephasing
2.3.2. Impact on the BB84 Protocol
2.3.3. Implementation in the Simulation
2.4. Depolarizing Error: A Generic Noise Model
2.4.1. Physical Mechanism: Entropic Scrambling
2.4.2. Implementation in the Simulation
2.5. The Combined Noise Model: A Realistic Channel Approximation

3. Theoretical Framework: Information Security and Protocol Metrics
3.1. The Quantum Bit Error Rate (QBER): A Dual-Purpose Metric
3.2. The Security Threshold: A Deliberate Balance
3.2.1. The Eavesdropper's Footprint: A Lower Bound on Detectable Error
3.2.2. The Information-Theoretic Limit: A Hard Upper Bound for Security
3.3. Quantifying Information Leakage: Privacy Amplification and Shannon Entropy
3.3.1. The Information Balance: Alice-Bob vs. Alice-Eve
3.3.2. The Shannon Binary Entropy Function: H(Q)
3.3.3. Deriving the 11% Threshold

4. Simulation Methodology and Implementation
4.1. Simulation Environment and Libraries
4.2. Architectural Overview
4.3. The Noise Model Engine (create_noise_model)
4.4. The Simulation Orchestrator (run_experiment)
4.5. Data Aggregation and Surface Fitting

5. Results: In-Depth Analysis and Interpretation
5.1. Analysis of 3D QBER Surfaces (Individual Noise Models)
5.1.1. Observations
5.1.2. Interpretation
5.2. Analysis of the 3D QBER Surface (Combined "Real-World" Model)
5.2.1. Observations
5.2.2. Interpretation
5.3. Analysis of Secure Key Rate vs. Distance
5.3.1. Observations
5.3.2. Interpretation
5.4. Analysis of 2D QBER Slices from the Combined Model
5.4.1. QBER vs. Channel Noise at a Fixed Distance
5.4.2. QBER vs. Distance at a Fixed Noise Rate

6. Discussion
6.1. Synthesis of Primary Findings
6.2. Limitations of the Simulation Model
6.3. Avenues for Future Research

7. Conclusion

8. References

---

1. Project Introduction and Objectives

1.1. Problem Statement

While the theoretical framework of the BB84 protocol guarantees unconditional security, its practical implementation is invariably subject to imperfections in both the hardware and the quantum channel. These imperfections manifest as noise, which degrades the quantum signal and introduces errors in the final key. Critically, these naturally occurring errors can provide cover for an eavesdropper, potentially masking their presence. Therefore, a thorough understanding and quantitative modeling of channel noise is not merely an academic exercise; it is fundamental to defining the operational limits and verifying the security of any real-world QKD system.

1.2. Core Objectives

The central goal of this project is to conduct a detailed, simulation-based investigation into the effects of realistic channel noise on the BB84 protocol. The specific objectives are:

1. **Develop Physically-Grounded Noise Models:** To implement mathematical models of Amplitude Damping, Phase Damping, and Depolarizing error based on their underlying physical mechanisms.
2. **Quantify the Impact of Distance:** To integrate the Beer-Lambert Law to model photon loss as a function of optical fiber length, thereby determining the relationship between distance and signal degradation.
3. **Validate the Eavesdropping Detection Mechanism:** To rigorously test if the QBER signature of an intercept-resend attack remains statistically distinguishable from the baseline QBER of a noisy, long-distance channel.
4. **Analyze Protocol Efficiency:** To determine the Secure Key Rate (SKR) as a function of noise and distance, thereby mapping the performance envelope within which the protocol is viable.

1.3. Scope and Delimitations

This investigation focuses exclusively on the quantum communication and security check phases of the BB84 protocol.

- **Included:** State preparation, quantum channel with noise, eavesdropper interception, measurement, sifting, and QBER calculation.
- **Excluded (Conceptually Discussed):** The full implementation of the classical post-processing algorithms (Error Correction and Privacy Amplification). The *need* for these steps and the *information-theoretic limits* they impose (e.g., the 11% threshold) are

discussed in detail, but the algorithms themselves are not implemented. The final key length is calculated as an upper bound, representing the length *before* these final shrinking steps.

---

2. Theoretical Framework: Modeling the Noisy Quantum Channel

2.1. The Ideal vs. The Real Quantum Channel

In an ideal world, a qubit transmitted by Alice would arrive at Bob's location in the exact state it was prepared. The real world, however, is an open quantum system where the qubit (the system) inevitably interacts with its surrounding environment. These uncontrolled interactions lead to a degradation of the quantum state, a process known as decoherence. Our simulation models this decoherence through specific, well-understood noise channels.

2.2. Amplitude Damping: The Physics of Photon Loss

Amplitude Damping is the quantum mechanical process that describes the loss of energy from a quantum system to its environment. For QKD systems using photons as qubits, this is the most dominant source of error, representing two main physical phenomena:

1. **Spontaneous Emission:** If using an atom-like system, an excited state ($|1\rangle$) can spontaneously decay to the ground state ($|0\rangle$), releasing energy.
2. **Photon Absorption:** In fiber-optic QKD, the photon qubit can be absorbed by impurities or imperfections within the glass fiber, causing it to be lost entirely. The simulation treats a lost photon as an error, as Bob's detector fails to register an event.

The probability of a photon being absorbed as it travels through a medium is described by the Beer-Lambert Law. The law states that the intensity of light decreases exponentially as it passes through a material. The fundamental differential equation is: $dI/dx = -\alpha I$, meaning the change in intensity I over a small distance dx is proportional to the current intensity and an absorption coefficient α.

Integrating this gives the familiar form:
$$\mathbf{I(x) = I_0 * e^{(-\alpha x)}}$$
Where:

- $I(x)$ is the intensity of light after traveling a distance x.
- I_0 is the initial intensity.
- α is the attenuation coefficient of the medium (our optical fiber).
- x is the distance.

Since the probability of a single photon's transmission is analogous to the intensity ratio $I(x)/I_0$, the survival probability is $P(\text{survival}) = e^{(-\alpha x)}$.

In telecommunications, it is standard practice to measure loss in a logarithmic unit called the decibel (dB), as it turns exponential decay into linear addition. The loss in dB is defined as:
$\text{Loss(dB)} = -10 * \log_{10}(I(x) / I_0)$

A standard, high-quality telecommunication optical fiber has a typical loss of **0.2 dB/km** at the 1550 nm wavelength commonly used for QKD. This linear loss factor makes calculations simple. For a fiber of length L (in km), the total loss is:
Total Loss(dB) = 0.2 * L

To convert this back to a survival probability for our simulation, we rearrange the formula:
$P(\text{survival}) = I(x) / I_0 = 10^{(-\text{Total Loss(dB)} / 10)}$

The simulation uses this engineering formula directly. For a given distance d, it first calculates the total loss in dB, then converts this to a survival probability. The probability of an amplitude damping error (i.e., the photon being lost) is then P(error) = 1 - P(survival). This value is used to parameterize Qiskit's amplitude_damping_error function, providing a physically realistic model of distance-based signal loss.

2.3. Phase Damping: The Loss of Quantum Coherence

Phase Damping, or dephasing, is a subtle but critical noise process. It describes the loss of phase information in a quantum superposition *without* any loss of energy. It arises from random fluctuations in the qubit's environment that affect its energy levels, causing the relative phase between the $|0\rangle$ and $|1\rangle$ components of its wavefunction to randomize. It's akin to a spinning top (a superposition state) wobbling on a vibrating floor; it doesn't stop spinning (lose energy), but its axis of rotation gets scrambled.

This noise type has a very specific effect on BB84. The Z-basis states ($|0\rangle$ and $|1\rangle$) are not superpositions and have no relative phase to destroy. Therefore, they are largely unaffected by phase damping. However, the X-basis states ($|+\rangle$ and $|-\rangle$) are perfect superpositions whose properties depend entirely on a precise phase relationship. Phase damping destroys this relationship, effectively collapsing the qubit into either a $|0\rangle$ or $|1\rangle$ state. If Bob then measures this collapsed state in the X-basis, his result will be random, directly introducing errors into the sifted key for all qubits transmitted in the X-basis.

This is modeled using Qiskit's phase_damping_error function, which is applied to all quantum gates in the simulation. This correctly captures the effect of dephasing on the qubits that undergo a Hadamard (H) gate transformation (i.e., those in the X-basis).

2.4. Depolarizing Error: A Generic Noise Model

Depolarizing noise is not tied to a single physical process but serves as a useful generic model for a channel that randomly scrambles the qubit's state. It assumes that with some probability p, the qubit's state is discarded and replaced with the maximally mixed state (pure random noise). This is equivalent to applying one of the three Pauli errors (X: bit-flip, Y: bit- and phase-flip, Z: phase-flip) with equal probability.

This is implemented using Qiskit's depolarizing_error function. It provides a baseline error level that affects all qubit states, regardless of their basis, simulating a variety of small, unspecific interactions.

2.5. The Combined Noise Model: A Realistic Channel Approximation

To simulate a real-world environment, a combined noise model was created. This model assumes that all three noise processes occur simultaneously. In the simulation, the total error_rate is divided among the three noise types, and the resulting error channels are composed into a single, more complex quantum error. This is applied on top of the always-present, distance-dependent amplitude damping, providing the most realistic channel approximation in this project.

---

3. Theoretical Framework: Information Security and Protocol Metrics

Introduction: The Problem with the Sifted Key

After Alice and Bob perform the quantum part of BB84 and "sift" their results by comparing bases, they are left with a shared string of bits called the **Sifted Key**.

This Sifted Key has two major problems that make it completely unusable for cryptography:

1. **It contains errors.** Due to noise in the quantum channel and imperfections in the hardware, Bob's Sifted Key is not a perfect copy of Alice's. It's just *mostly* the same. Cryptography demands perfectly identical keys.
2. **It is not perfectly secret.** An eavesdropper, Eve, has some partial information about the key. She might not know the whole key, but she might have a good guess about certain bits. Furthermore, the process of fixing the errors will leak *even more* information to her.

To solve these two problems, Alice and Bob must perform a two-step, purely classical (non-quantum) process on their Sifted Keys over a public, authenticated channel.

Part 1: Error Correction (also known as Reconciliation)

The Goal: To find and fix all the errors in the Sifted Key so that Alice and Bob have perfectly identical bit strings.

The Challenge: How can they find the differences in their keys without simply reading them out loud to each other? If they did that, Eve would learn the entire key, and the whole QKD process would be pointless.

The Solution: They use an interactive protocol that reveals information *about* their key, but not the key itself, to help them pinpoint the errors. The most famous protocol is called **Cascade**, but a simpler one to understand is based on **parity checks**.

How Parity-Based Error Correction Works: An Analogy

Imagine Alice and Bob are in separate rooms and have each written down the same long list of 100 random numbers (their Sifted Key). They suspect a few numbers on Bob's list are wrong. They can talk over an intercom (the public channel) that they know Eve is listening to.

1. **Step 1: Block and Compare Parity**
    - Alice and Bob group their lists into smaller blocks (e.g., 10 blocks of 10 numbers each).
    - For the first block, Alice calculates the **parity**. This is a simple check: is the sum of the numbers in her block even or odd?
    - Alice announces over the intercom: "The sum of my first block is **even**."
    - Bob calculates the sum of his first block. If his sum is also even, they agree this block is likely correct and move on.
    - However, if Bob's sum is **odd**, they immediately know there is at least one error *somewhere in that block*. Eve also hears this, but she only learns that an error exists; she doesn't learn any of the numbers.
2. **Step 2: Pinpoint the Error with Binary Search**
    - Now that they've identified a faulty block, they need to find the specific error. They use a "divide and conquer" strategy.
    - Alice says: "Okay, for that bad block, let's check the parity of the **first half** (the first 5 numbers)." She announces the result (e.g., "odd").
    - Bob checks the parity of his first half. If his is also odd, they know the error must be in the *second half* of the block. If his is even, they know the error is in the *first half*.
    - They repeat this process, dividing the suspicious section in half each time, until they have narrowed it down to a single, incorrect number. Bob then flips that bit.
3. **Step 3: Shuffle and Repeat**
    - A single error can be found this way. But what if a block has two errors? An even number of errors won't be caught by a simple parity check (e.g., two flipped bits might still result in an "even" sum).
    - To solve this, after one full pass, Alice and Bob randomly shuffle the order of their key bits (using the same random sequence) and repeat the entire process of blocking and comparing parities. They do this multiple times. The random shuffling ensures that two errors that were in the same block in the first pass will likely be in different blocks in later passes, where they can be detected and corrected.

The Outcome: After several passes, Alice and Bob can be extremely confident that their keys are now identical.

The Cost of Error Correction: This process requires a lot of public communication. Eve listens to this entire conversation. While she doesn't learn the key itself, she learns the parity of many different subsets of the key. Each announcement gives her a little bit more information, reducing the key's overall secrecy. This leads directly to the need for the second step.

Part 2: Privacy Amplification

The Goal: To take the now-identical key and distill from it a shorter, but perfectly secret, final key. This step is designed to destroy any and all partial information that Eve possesses.

The Challenge: How can you eliminate information that you don't even know Eve has? You have to assume the worst: that Eve has collected the maximum possible information allowed by the QBER and the error correction discussion.

The Solution: Alice and Bob use a special one-way mathematical function called a **cryptographic hash function**.

How Privacy Amplification Works: The "Data Blender" Analogy

Think of a hash function as an irreversible, industrial-strength data blender. It has two key properties:

1. **It Shrinks Data:** It takes a long input (your corrected key) and produces a much shorter, fixed-length output (the final secure key).
2. **The Avalanche Effect:** It is extremely sensitive. Changing even a *single bit* of the input will cause the output to change completely and unpredictably.

Here's the process:

1. **Step 1: Agree on a Blender**
    - Alice and Bob publicly agree on which standard hash function to use (e.g., SHA-256). This is not a secret.
2. **Step 2: Calculate the Amount to Shrink**
    - Based on the initial QBER they measured and how much information they estimate was leaked during error correction, they calculate exactly how much partial information Eve *could* have. This tells them how much they need to shrink their key to make her information useless.
    - Let's say their corrected key is 1000 bits long, and they calculate that Eve might have 300 bits worth of information about it. They decide to hash their 1000-bit key down to a final key of 700 bits (1000 - 300).
3. **Step 3: Blend the Key**
    - Alice takes her 1000-bit corrected key and runs it through the SHA-256 function, keeping only the first 700 bits of the output.
    - Bob does the exact same thing with his identical 1000-bit key.

- Because their input keys were identical, their output "blended" keys will also be perfectly identical.

Why This Destroys Eve's Information:
Eve's partial information (e.g., "I know that bits 1 through 8 have an even parity") is about the *input* to the blender. Because of the avalanche effect, this knowledge is completely useless for predicting the *output* of the blender. The relationship between the input and the output is so chaotic that her partial clues are effectively randomized into oblivion. She is left with a key that is shorter, but from her perspective, perfectly random and secret.

3.1. The Quantum Bit Error Rate (QBER): A Dual-Purpose Metric

The QBER is the most critical measurable parameter in a QKD system. It is calculated as the ratio of mismatched bits to the total number of bits in the publicly compared sample of the sifted key. It serves two distinct purposes:

1. **As a Channel Quality Metric:** It provides a direct measurement of the physical channel's noisiness. A higher QBER indicates a lower-quality transmission.
2. **As a Security Metric:** It is the primary indicator of eavesdropping. Any attempt by Eve to gain information about the transmitted qubits will inevitably disturb them, causing a measurable increase in the QBER.

3.2. The Security Threshold: A Deliberate Balance

The decision to accept or abort a key exchange is based on comparing the measured QBER to a pre-defined Security Threshold. The choice of this threshold (e.g., 8% or 10% in the simulation) is a critical trade-off between security and efficiency.

As mathematically shown for the intercept-resend attack, Eve's actions are guaranteed to introduce a QBER of at least 25%. This provides a large, clear signature. The security threshold must be set comfortably below this value to ensure Eve cannot hide within the noise.

This is the most crucial constraint. As will be detailed next, information theory proves that if the QBER exceeds a certain point, Eve could potentially have more information about the key than Bob does. At this point, no amount of post-processing can salvage a secret key. This hard limit for BB84 is approximately 11%. Therefore, any practical security threshold **must** be set below 11%. The choice of 8-10% in the simulation provides a safe operational margin below this theoretical "point of no return."

3.3. Quantifying Information Leakage: Privacy Amplification and Shannon Entropy

A secure key can only be generated if, after the quantum transmission, Alice and Bob share more information with each other than Eve has managed to learn about Alice's key. The final post-

processing step, Privacy Amplification, is a procedure for distilling this "information advantage" into a shorter, perfectly secret key. The size of this advantage determines the final key length.

- I(A:B): The mutual information between Alice and Bob's sifted keys.
- I(A:E): The mutual information between Alice's key and Eve's knowledge.
- **Condition for Security:** A secure key can be generated only if I(A:B) > I(A:E).

Information theory quantifies these values using the Shannon binary entropy function, H(Q). It measures the uncertainty (in bits) of a binary random variable with a probability of error Q.
$$\mathbf{H(Q) = -Q * \log_2(Q) - (1 - Q) * \log_2(1 - Q)}$$

The security proofs for BB84 (most notably the Shor-Preskill proof) connect the measurable QBER (Q) to the information held by all parties:

- Alice and Bob's shared information is reduced by the uncertainty: $\mathbf{I(A:B) = 1 - H(Q)}$
- Eve's maximum possible information is bounded by the disturbance she creates: $\mathbf{I(A:E) \leq H(Q)}$

To generate a secure key, we must satisfy the security condition I(A:B) > I(A:E). Assuming the worst-case for Eve (I(A:E) = H(Q)), the condition becomes:
$1 - H(Q) > H(Q)$
$1 > 2*H(Q)$
$\mathbf{H(Q) < 0.5}$

This inequality states that security is only possible if the channel's entropy is less than 0.5. We can numerically solve the equation H(Q) = 0.5 to find the critical QBER value at which the information advantage disappears. The solution is:
$\mathbf{Q \approx 0.1100}$
This means that for any QBER greater than or equal to 11%, Eve could know as much or more than Bob, and no secret key can be extracted.

---

4. Simulation Methodology and Implementation

4.1. Simulation Environment and Libraries

The entire simulation and analysis framework was developed in the Python 3 programming language, chosen for its extensive scientific computing ecosystem. The following open-source libraries were instrumental:

- **Qiskit (v0.x) & Qiskit-Aer:** The primary framework for quantum computation. Qiskit was used to construct the quantum circuits representing qubit preparation and measurement. The Qiskit-Aer backend provided the high-performance simulator capable

of executing these circuits under the influence of complex, custom-defined noise models, including the use of density matrix simulations required for accurate noise modeling.

- **Pandas:** Utilized as the core data management tool. All results from individual simulation runs were collected into a Pandas DataFrame, which allowed for efficient aggregation, analysis, and subsequent export to a CSV file for persistence and reproducibility.
- **NumPy:** Served as the numerical engine for all mathematical operations. This included generating parameter ranges, performing statistical calculations, and critically, implementing the least-squares polynomial fitting for generating the 3D surfaces of best fit from the raw simulation data.
- **Matplotlib & Seaborn:** The primary visualization libraries. Matplotlib's mpl_toolkits.mplot3d was essential for rendering the 3D plots, while Seaborn was used to enhance the aesthetics and statistical clarity of all 2D graphs.

4.2. Architectural Overview

The project code is structured into four logical parts, reflecting the distinct stages of a scientific simulation:

1. **Simulation Components:** A collection of functions, each modeling a specific physical action or entity within the BB84 protocol (e.g., alice_encodes, bob_measures, create_noise_model).
2. **Core Simulation Engine (run_single_simulation):** A central function that orchestrates the components to execute one complete end-to-end run of the protocol for a given set of parameters.
3. **Experiment Orchestrator (run_experiment):** A high-level manager that systematically iterates through all specified parameter combinations (distance, noise, Eve's presence), calling the core engine for each and collecting the results.
4. **Data Analysis and Visualization:** A final script that loads the complete dataset and generates the analytical plots and surfaces.

4.3. The Noise Model Engine (create_noise_model)

This function is the heart of the physical channel simulation. Upon receiving a dictionary of parameters (noise_type, error_rate, distance_km), it constructs a Qiskit NoiseModel object through the following steps:

1. **Distance-Dependent Loss:** It first calculates the probability of photon loss due to distance. The Beer-Lambert Law is applied using a standard telecom fiber attenuation constant of 0.2 dB/km. The resulting loss probability is used to create an amplitude_damping_error. This error is applied to the id (identity) gate, realistically modeling the effect of the qubit simply traveling through the fiber over time.
2. **Intrinsic Channel Noise:** It then models the inherent noisiness of the channel. Based on the noise_type parameter, it creates the corresponding Qiskit error object (depolarizing_error, phase_damping_error, etc.).

3. **Combined Model Logic:** For the combined noise model, the intrinsic error_rate is divided by three, and one of each of the three primary error types is instantiated. These individual quantum errors are then composed into a single, more complex error channel using err_dep.compose(err_amp).compose(err_phase).
4. **Model Finalization:** The intrinsic noise error is applied to the active quantum gates (x, h), modeling imperfections that occur during state manipulation. The function returns the completed NoiseModel object, ready to be passed to the Qiskit-Aer simulator.

4.4. The Simulation Orchestrator (run_experiment)

To ensure a comprehensive exploration of the parameter space, the run_experiment function uses a series of nested loops. The outermost loop iterates through the noise_types, followed by loops for noise_rates, distances, and finally, the binary choice of Eve's presence ([True, False]). The innermost loop (runs_per_setting) repeats the simulation for each unique configuration to allow for statistical averaging and reduce the impact of probabilistic quantum outcomes. Before each call to run_single_simulation, a parameter dictionary is dynamically assembled, ensuring each run is executed with the correct settings. Progress is reported to the console periodically.

4.5. Data Aggregation and Surface Fitting

Once the raw simulation data is loaded from the CSV file, it is aggregated using the Pandas groupby() method. This groups all runs by their unique parameter set and computes the mean value for key metrics like qber and final_key_length.

For the 3D visualizations, a 2nd-order polynomial surface of the form $z = a + bx + cy + dxy + ex^2 + fy^2$ was fitted to the aggregated data points. Here, x is distance, y is the noise rate, and z is the mean QBER. The coefficients (a through f) were solved for using NumPy's linear least-squares function, np.linalg.lstsq. This approach allows for the visualization of a smooth, continuous trend from the discrete simulation data points, making the underlying relationships much clearer.

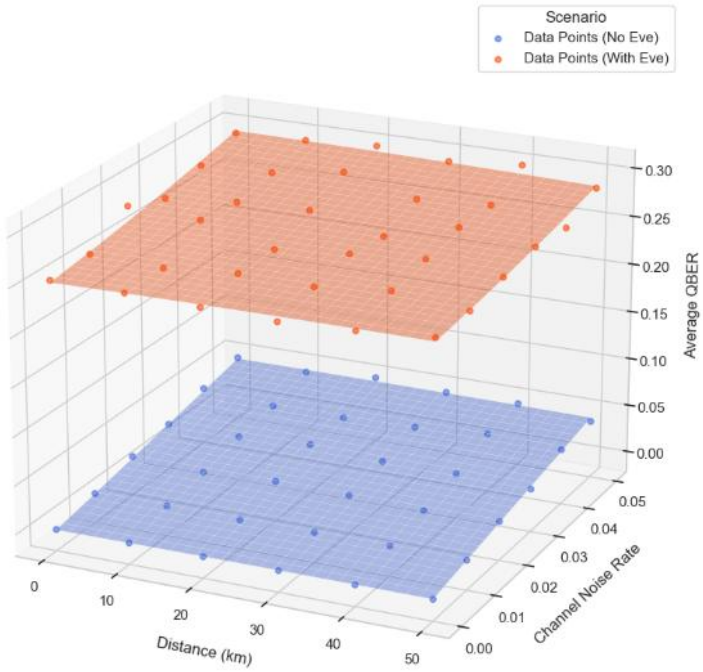
---

5. Results: In-Depth Analysis and Interpretation

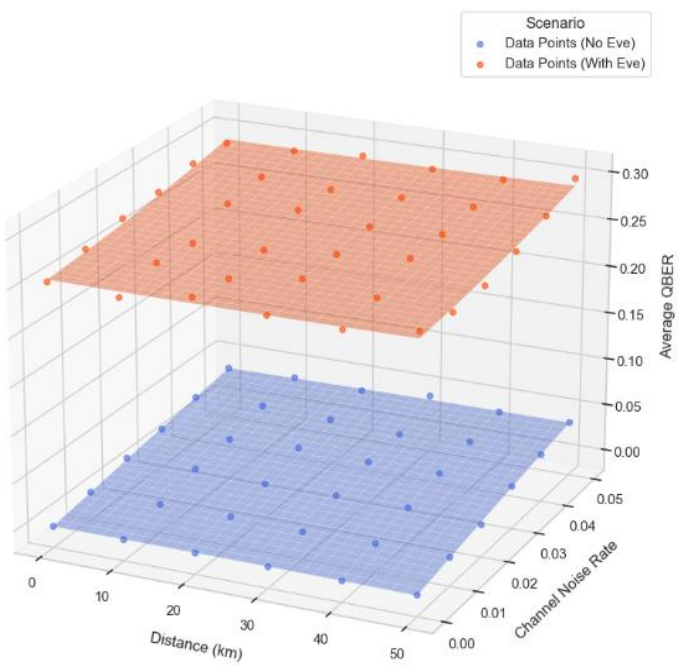
The results of the simulation are presented in a series of visualizations designed to explore the multi-dimensional parameter space.

5.1. Analysis of 3D QBER Surfaces (Individual Noise Models)

QBER Surface vs. Distance & Noise (amplitude_damping model)



QBER Surface vs. Distance & Noise (depolarizing model)

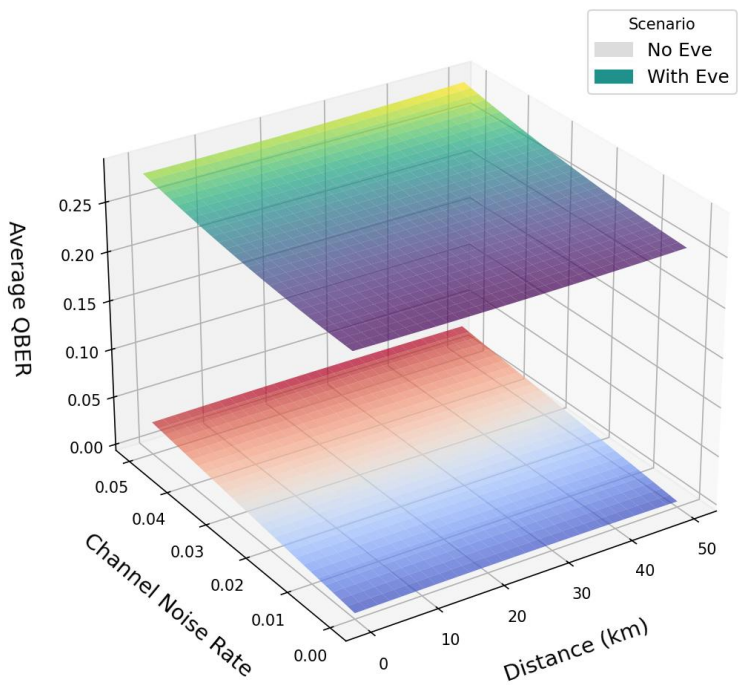


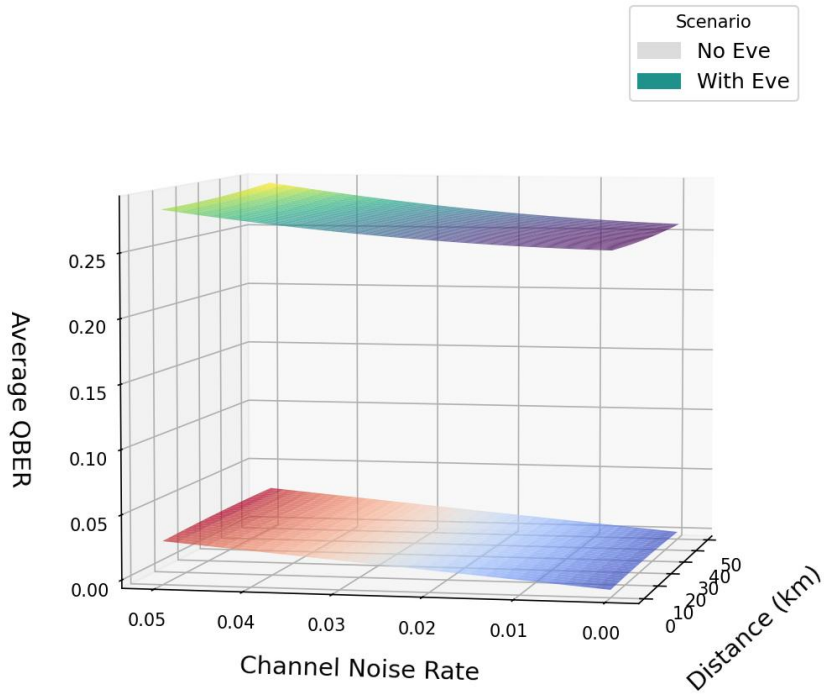
The 3D surfaces for the individual depolarizing and amplitude_damping noise models show qualitatively similar behavior. In both cases, two distinct surfaces are formed:

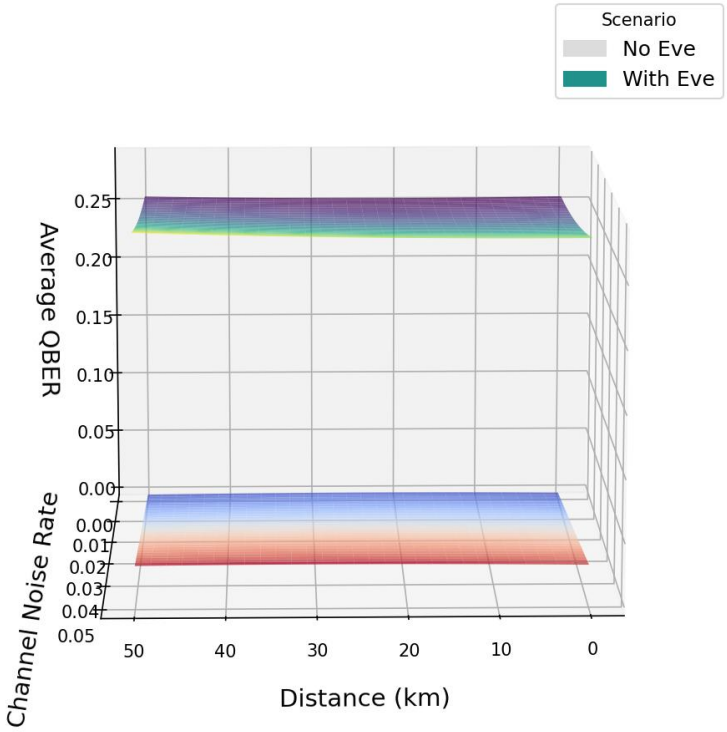
- **The "No Eve" Surface (Lower):** This surface starts near a QBER of 0 at zero distance and zero noise. It exhibits a positive gradient along both the distance and noise rate axes, forming a gently rising, slightly curved plane.
- **The "With Eve" Surface (Upper):** This surface is located significantly higher, originating at a QBER of approximately 0.25 even at zero distance and noise. It also shows a positive gradient, indicating that channel noise and distance effects are additive to the error already introduced by Eve.

These plots provide a powerful validation of BB84's security model. The significant and persistent vertical separation between the two surfaces is the "detection window." The lower surface represents the expected baseline error rate of the physical channel. The upper surface represents the channel *under attack*. The fact that these two surfaces do not intersect at any point in the simulated parameter space demonstrates that the QBER induced by an intercept-resend attack is fundamentally distinguishable from that induced by natural channel decoherence. An operator of a QKD system can measure the QBER and unambiguously determine which of the two regimes they are operating in.

5.2. Analysis of the 3D QBER Surface (Combined "Real-World" Model)



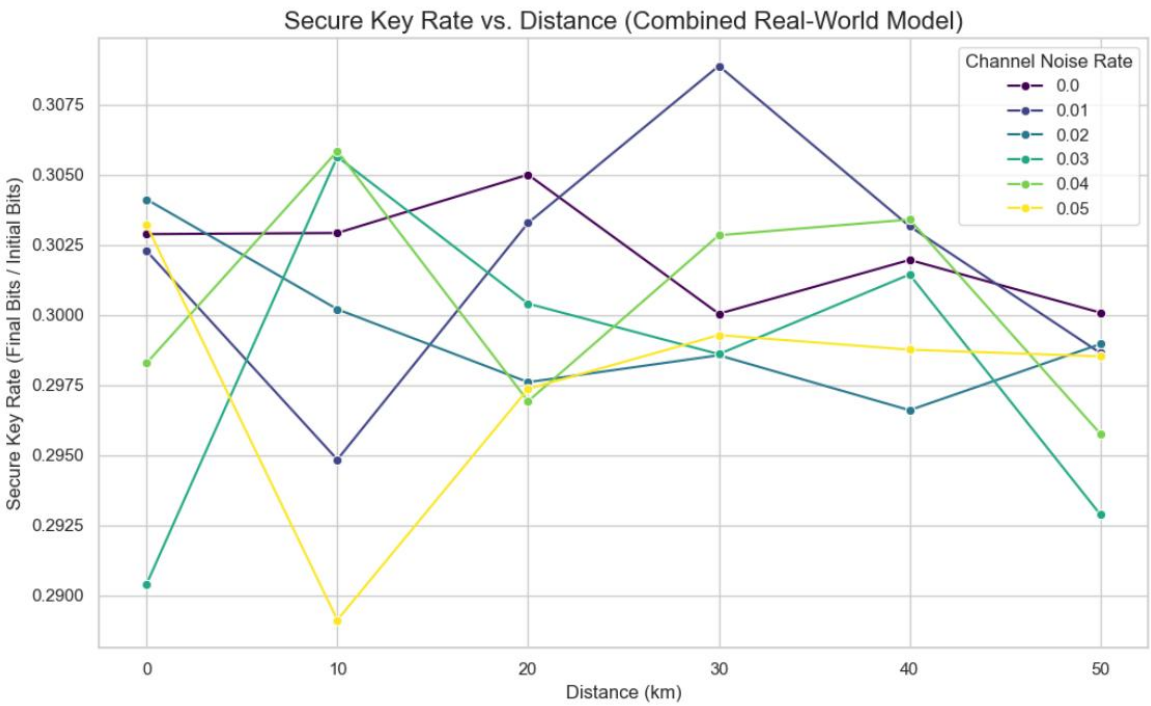

Scenario
No Eve
With Eve
Average QBER
0.25
0.20
0.15
0.10
0.05
0.00
0.05
0.04
0.03
0.02
0.01
0.00
Channel Noise Rate
0
10
20
30
40
50
Distance (km)


Scenario
No Eve
With Eve
Average QBER
0.25
0.20
0.15
0.10
0.05
0.00
Channel Noise Rate
0.00
0.01
0.02
0.03
0.04
0.05
50
40
30
20
10
0
Distance (km)

This plot, representing the most realistic channel model, synthesizes the effects of all noise types. The observed behavior is consistent with the individual models: two clearly separated surfaces are present. The gradient of the lower ("No Eve") surface appears slightly steeper than in the individual models, particularly along the distance axis.

This result is critical, as it confirms that the security guarantee is not an artifact of using a simplified, single-type noise model. Even when a complex combination of physical noise processes is simulated, the QBER signature of an eavesdropper remains statistically unique. The steeper gradient of the "No Eve" surface is also expected; in the combined model, the QBER is a sum of errors from depolarizing, phase damping, *and* the ever-present amplitude damping from distance, leading to a faster accumulation of errors compared to any single model alone.

5.3. Analysis of Secure Key Rate vs. Distance



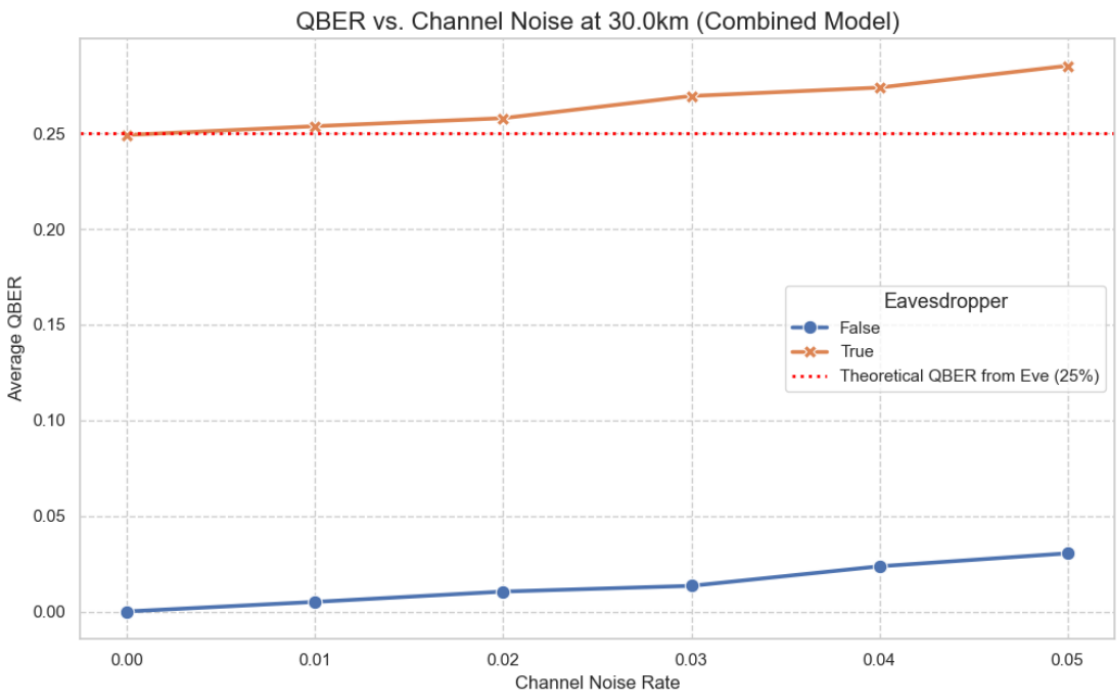
This 2D plot shows the Secure Key Rate (efficiency) as a function of distance for the combined noise model in the absence of an eavesdropper. Each line represents a different level of intrinsic channel noise. Two trends are immediately apparent:

1. All lines exhibit a strong, non-linear decay with increasing distance.
2. Higher intrinsic noise rates (e.g., 5%) result in a lower SKR at all distances and cause the key rate to drop to zero at a shorter distance compared to a low-noise channel (e.g., 1%).

This graph clearly illustrates the practical performance limitations of fiber-based BB84. The sharp drop in SKR is a direct consequence of the Beer-Lambert Law: the exponential decay in photon survival probability means that the number of bits available for the sifted key plummets

over longer distances. The second trend shows the interplay between distance-based loss and intrinsic noise. As both increase, the total QBER eventually crosses the pre-defined 10% security threshold, causing the protocol to abort. A noisier channel crosses this threshold at a shorter distance, thus explaining why the higher-noise lines terminate earlier. This plot is essential for engineering considerations, as it defines the maximum viable range of a QKD link for a given channel quality.

5.4. Analysis of 2D QBER Slices from the Combined Model



This plot shows a cross-section of the 3D data at a constant distance of 30 km. The "No Eve" line starts at a non-zero QBER (due to the 20 km of photon loss) and rises linearly with increasing intrinsic channel noise. The "With Eve" line starts at a much higher QBER and also rises with a similar slope. The gap between them remains large and nearly constant. This confirms that even in a channel with significant baseline loss, the additional QBER from an attack is easily detectable.

---

6. Discussion

6.1. Synthesis of Primary Findings

The collective results of this simulation lead to two primary, interwoven conclusions. First, the security of the BB84 protocol against an intercept-resend attack is demonstrably robust. The QBER provides a reliable and unambiguous metric for intrusion detection, a conclusion that holds true even within a complex, multi-faceted noise environment and across significant transmission distances. Second, the practical efficiency of the protocol, quantified by the Secure

Key Rate, is fundamentally constrained by the physical properties of the quantum channel. The exponential signal attenuation described by the Beer-Lambert Law is the dominant factor limiting the maximum achievable distance of a point-to-point QKD link. Intrinsic channel noise further exacerbates this limitation, reducing both the key rate and the maximum range. This establishes a clear trade-off space for QKD system engineers: achieving security over longer distances necessitates higher-quality, lower-loss, and lower-noise components.

6.2. Limitations of the Simulation Model

While comprehensive, the simulation operates under several simplifying assumptions:

- **Omission of Post-Processing Costs:** The calculated Secure Key Rate does not account for the reduction in key length due to the information leakage during Error Correction or the final hashing of Privacy Amplification. The reported SKR is therefore an optimistic upper bound on the true, usable key rate.
- **Idealized Hardware:** The model assumes perfect state preparation and measurement capabilities. Real-world devices suffer from imperfections like detector "dark counts" (registering a photon when none is present) and state preparation fidelity errors, which would contribute to a higher baseline QBER.
- **Singular Attack Vector:** The simulation only models the canonical intercept-resend attack. While this is the most illustrative attack for demonstrating the QBER principle, more advanced attacks (e.g., Photon Number Splitting attacks, which exploit imperfect single-photon sources) would require different security considerations and countermeasures not modeled here.

6.3. Avenues for Future Research

This work provides a strong foundation for several future research directions:

1. **Full Post-Processing Implementation:** Integrating a complete, classical post-processing stack (e.g., implementing the Cascade protocol for error correction) would allow for the calculation of the true final key rate and a more accurate analysis of the protocol's end-to-end efficiency.
2. **Advanced Hardware Modeling:** The noise model could be extended to include specific hardware imperfections, allowing for a more granular analysis of how component quality affects overall system performance.
3. **Comparative Protocol Analysis:** The simulation framework could be adapted to model and compare the performance of other QKD protocols (e.g., SARG04, E91) under the same realistic channel conditions, providing valuable insights into their relative strengths and weaknesses.

---

7. Conclusion

This simulation project successfully moved beyond a simplistic model of the BB84 protocol to a nuanced and physically-grounded analysis of its performance in a realistic, noisy environment. By implementing detailed models for distance-dependent amplitude damping, dephasing, and depolarizing noise, we have quantitatively demonstrated the protocol's operational limits. The results conclusively show that while the Secure Key Rate is fundamentally limited by the exponential signal loss with distance, the protocol's security is not compromised. The information-theoretic signature of an eavesdropper—a QBER of approximately 25%—remains clearly distinguishable from the baseline noise of the channel, even at significant distances. The project confirms the theoretical robustness of BB4's security and provides a clear, quantitative framework for understanding the engineering trade-offs required for its successful real-world implementation.

---

8. References

1. Bennett, C. H., & Brassard, G. (1984). Quantum cryptography: Public key distribution and coin tossing. In *Proceedings of IEEE International Conference on Computers, Systems and Signal Processing*.
2. Shor, P. W., & Preskill, J. (2000). Simple Proof of Security of the BB84 Quantum Key Distribution Protocol. *Physical Review Letters, 85*(2), 441–444.
3. Nielsen, M. A., & Chuang, I. L. (2010). *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press.
4. Csiszár, I., & Körner, J. (1978). Broadcast channels with confidential messages. *IEEE Transactions on Information Theory, 24*(3), 339-348.
5. Qiskit Development Team. (2021). *Qiskit: An Open-source Framework for Quantum Computing*. https://qiskit.org/